

Verification Environment

Open-Source Based

Technology Stack Overview

- Linux Environment
- Verification Tools
- Source code / Git

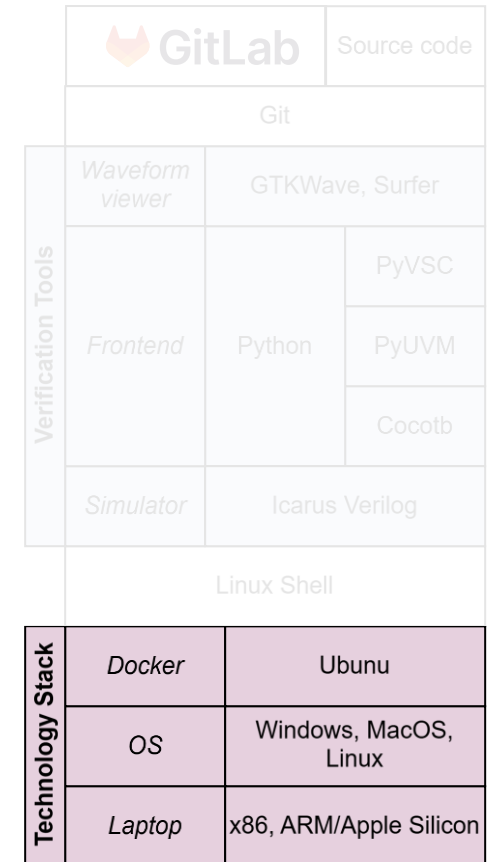
 GitLab		Source code	
Git			
Verification Tools	<i>Waveform viewer</i>	GTKWave, Surfer	
	<i>Frontend</i>	Python	PyVSC
			PyUVM
			Cocotb
	<i>Simulator</i>	Icarus Verilog	
Linux Shell			
Technology Stack	<i>Docker</i>	Ubunu	
	OS	Windows, MacOS, Linux	
	<i>Laptop</i>	x86, ARM/Apple Silicon	

Linux Environment



Linux Environment

- Toolchain runs in Linux
- In this course: Linux environment in Docker
 - **Docker**: virtualization platform
 - **Dockerfile**: Describes how to build a *Docker image*
 - **Docker Image**: Read-only template for env. with all dependencies
 - **Docker Container**: Isolated instance running an *image*
 - **Docker Volume**: Persistent storage, even if *container* is deleted



Navigating A Linux Shell

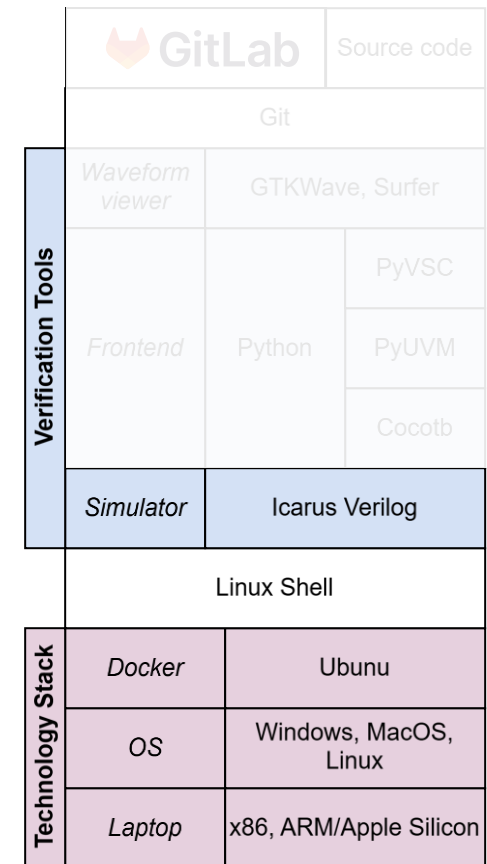
- Docker container has no desktop
 - Some knowledge of shell commands necessary
 - Standard shell in container: Bash
 - Installed editors:
 - Vim
 - Emacs
 - Nano
 - See
 - [verification_env_setup/verification_env_setup.pdf](#)
- Appendix 1 for more info on Terminal basics

Verification Tools	 Source code	
	Git	
	Waveform viewer	GTKWave, Surfer
	Frontend	PyVSC
		PyUVM
		Cocotb
Technology Stack	Simulator	Icarus Verilog
	Linux Shell	
	Docker	Ubuntu
	OS	Windows, MacOS, Linux
	Laptop	x86, ARM/Apple Silicon

Verification Tools

Simulators

- You may be used to proprietary tools like Xilinx Vivado or Intel Quartus. These perform simulation, synthesis, place-and-route and more in the same program, but also commercial tools like:
 - VCS, Synopsys
 - Questa, Siemens-EDA
 - Xcelium, Cadence
- This is not the general case when using Open-Source
- Must use multiple inter-operating tools to achieve same functionality
- Simulators: The biggest open-source projects are Icarus **Verilog** and **Verilator**



Icarus Verilog

- iverilog tool converts RTL to intermediate representation (IR)
- Not something you will hand-write

```
module dff (  
    input clk, reset,  
    input logic [7:0] d,  
    output logic [7:0] q  
);  
  
always_ff @(posedge clk) begin  
    if (reset) begin  
        q <= '0;  
    end else begin  
        q <= d;  
    end  
end  
  
endmodule: dff
```



```
:ivl_version "11.0 (stable)" "(v11_0-132-gb2f2414f4)";  
:ivl_delay_selection "TYPICAL";  
:vpi_time_precision + 0;  
:vpi_module "/home/jacob/nobackup/icarus/install/lib/ivl/system.vpi";  
:vpi_module "/home/jacob/nobackup/icarus/install/lib/ivl/vhdl_sys.vpi";  
:vpi_module "/home/jacob/nobackup/icarus/install/lib/ivl/vhdl_textio.vpi";  
:vpi_module "/home/jacob/nobackup/icarus/install/lib/ivl/v2005_math.vpi";  
:vpi_module "/home/jacob/nobackup/icarus/install/lib/ivl/va_math.vpi";  
:vpi_module "/home/jacob/nobackup/icarus/install/lib/ivl/v2009.vpi";  
S_0xffae10 .scope package, "$unit" "$unit" 2 1;  
    .timescale 0 0;  
S_0xffafc0 .scope module, "dff" "dff" 3 1;  
    .timescale 0 0;  
        .port_info 0 /INPUT 1 "clk";  
        .port_info 1 /INPUT 1 "reset";  
        .port_info 2 /INPUT 8 "d";  
        .port_info 3 /OUTPUT 8 "q";  
  
... (50 lines total)
```

Icarus Verilog

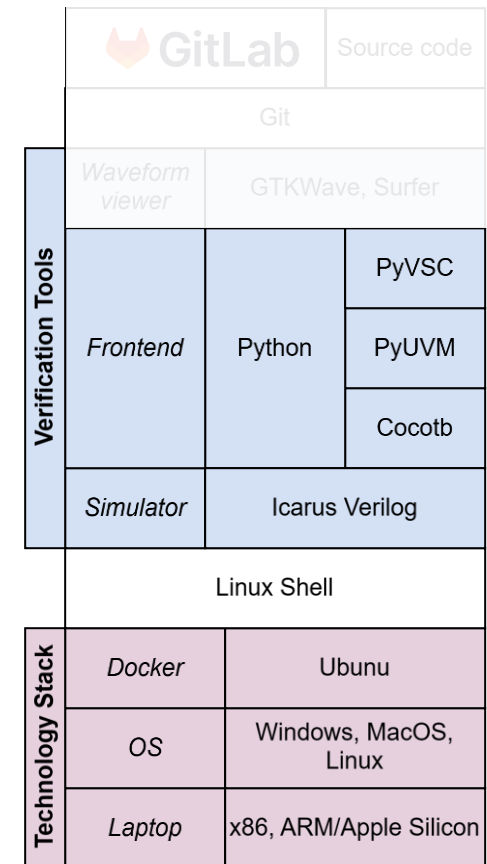
- To convert to VVP format
 - `iverilog -g2012 -o hello.vvp hello.sv`
 - `-g2012`: Use the SystemVerilog 2012 standard
 - `-o hello.vvp`: Set output file to hello.vvp
 - `hello.sv`: Use hello.sv as input file

```
module hello ();  
  initial begin  
    $display("Hello, world!");  
    $finish;  
  end  
endmodule: hello
```

```
$ iverilog -g2012 -o hello.vvp hello.sv #Convert to VVP  
$ vvp hello.vvp #Execute  
Hello, world!
```

Frontend

- You don't have to worry about the specifics of the simulator that you're using, or how to write wrapper modules
- Cocotb – Interaction with simulator from Python
- PyUVM – Python Implementation of UVM
- PyVSC – Constrained Randomization and Coverage



Waveform Viewers

- Waveforms are essential to debugging errors in hardware designs
- Graphical overview of the signals and their temporal relationships
- Open-source simulators do not have a GUI for viewing waveforms.
- Signal transitions are exported to Value Change Dump (VCD) and viewed with waveform viewer

		Source code
Git		
Verification Tools	Waveform viewer	GTKWave, Surfer
	Frontend	PyVSC
		PyUVM
		Cocotb
	Simulator	Icarus Verilog
Linux Shell		
Technology Stack	Docker	Ubuntu
	OS	Windows, MacOS, Linux
	Laptop	x86, ARM/Apple Silicon

Waveform viewers

- GTKWave

- The *de-facto* standard open-source waveform viewer
- Very powerful, fast and simple to use

- <https://github.com/gtkwave/gtkwave>

- Surfer

- New, promising tool for viewing waveforms
- Not as many features, but active open-source development

- <https://gitlab.com/surfer-project/surfer>

GitLab

GitLab

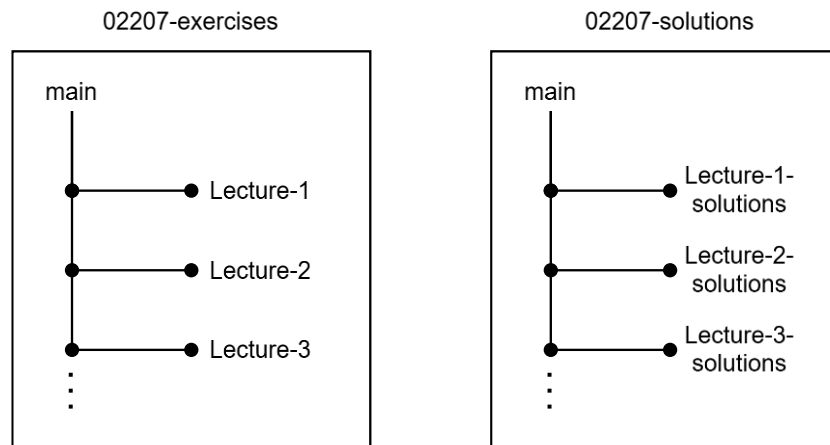
- Code for this course will be shared through DTU GitLab
 - <https://gitlab.gbar.dtu.dk/>
- New code will be released weekly
- Work locally or on your own GitLab repository
- See `verification_env_setup.pdf` Appendix 2 for more info on git

 GitLab		Source code
Git		
Verification Tools	Waveform viewer	GTKWave, Surfer
	Frontend	PyVSC
		PyUVM
		Cocotb
	Simulator	Icarus Verilog
Linux Shell		
Technology Stack	Docker	Ubuntu
	OS	Windows, MacOS, Linux
	Laptop	x86, ARM/Apple Silicon

Gitlab



- Two repositories:
 1. 02207-exercises
 2. 02207-solutions
- Each week's state preserved in a dedicated branch



 GitLab		Source code	
Git			
Verification Tools	<i>Waveform viewer</i>	GTKWave, Surfer	
	<i>Frontend</i>	Python	PyVSC
			PyUVM
			Cocotb
	<i>Simulator</i>	Icarus Verilog	
Linux Shell			
Technology Stack	<i>Docker</i>	Ubunu	
	OS	Windows, MacOS, Linux	
	<i>Laptop</i>	x86, ARM/Apple Silicon	

Linux Environment Setup

Linux setup

- Download `verification_env_setup.zip` from DTU learn
- Follow the steps in `verification_env_setup.pdf`

 GitLab		Source code
Git		
Verification Tools	Waveform viewer	GTKWave, Surfer
	Frontend	PyVSC
		PyUVM
		Cocotb
	Simulator	Icarus Verilog
Linux Shell		
Technology Stack	Docker	Ubuntu
	OS	Windows, MacOS, Linux
	Laptop	x86, ARM/Apple Silicon